

Session 11: Pointers Topics: Pointer declaration, dereferencing, pointer arithmetic. Example: Swap numbers using pointers. Trainer Demo: Use memory diagram to explain address referencing.

Tasks

- 1) Declare an integer variable called likes and a pointer variable called ptr Likes; assign likes a value, point ptr Likes to likes, and print both the value and the address stored in ptr Likes.**

```
#include <stdio.h>

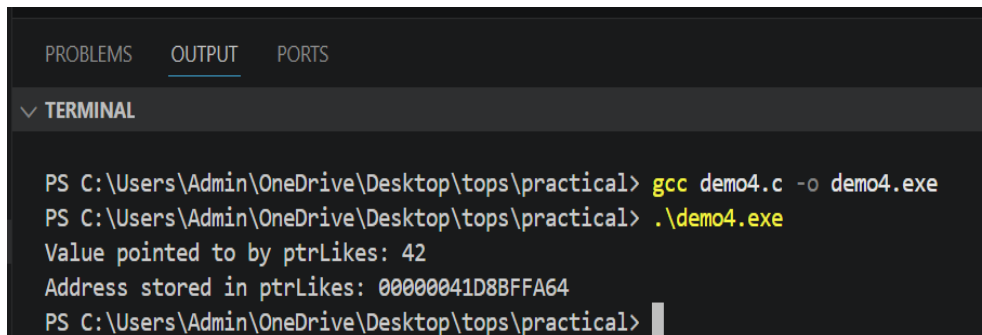
int main() {
    int likes;
    int *ptrLikes;

    likes = 42;
    ptrLikes = &likes;

    printf("Value pointed to by ptrLikes: %d\n", *ptrLikes);
    printf("Address stored in ptrLikes: %p\n", (void *)ptrLikes);

    return 0;
}
```

Output:



```
PROBLEMS  OUTPUT  PORTS
▼ TERMINAL

PS C:\Users\Admin\OneDrive\Desktop\tops\practical> gcc demo4.c -o demo4.exe
PS C:\Users\Admin\OneDrive\Desktop\tops\practical> .\demo4.exe
Value pointed to by ptrLikes: 42
Address stored in ptrLikes: 00000041D8BFFA64
PS C:\Users\Admin\OneDrive\Desktop\tops\practical> |
```

2) Write a function `swapPlaylistCounts(int *a, int *b)` that swaps the number of songs in two Spotify playlists using pointers, then call the function in main and print the swapped values.

```
#include <stdio.h>

void swapPlaylistCounts(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int playlist1Songs = 25; // e.g., "Chill Vibes" playlist
    int playlist2Songs = 40; // e.g., "Workout Mix" playlist

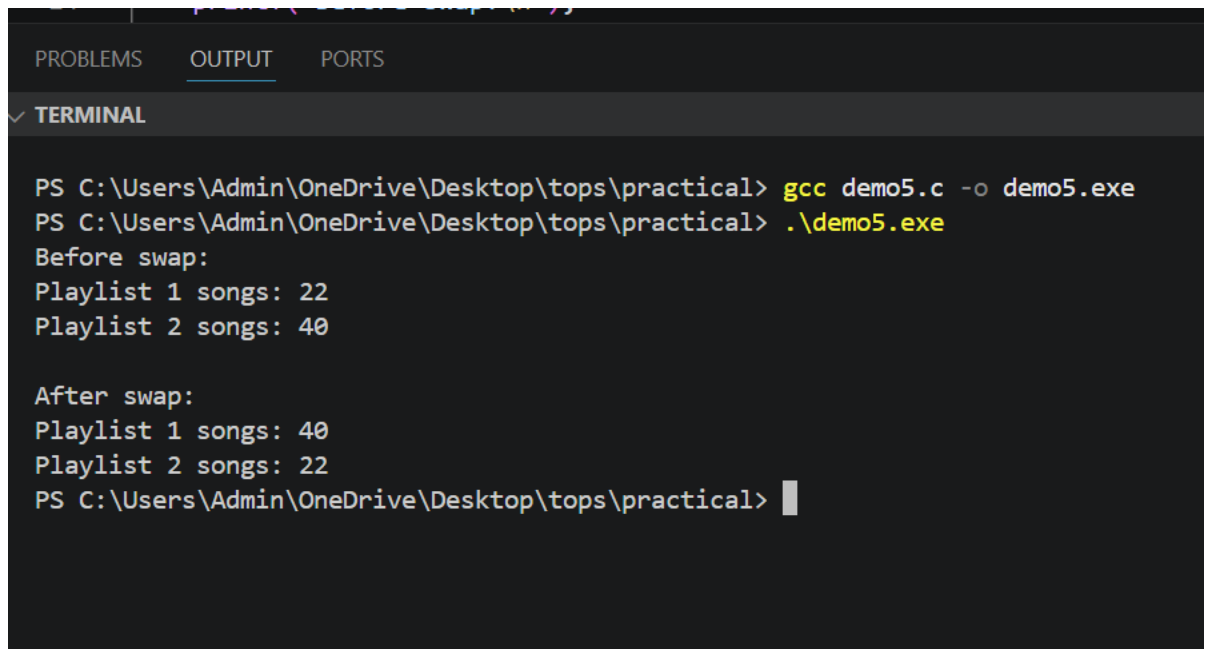
    printf("Before swap:\n");
    printf("Playlist 1 songs: %d\n", playlist1Songs);
    printf("Playlist 2 songs: %d\n", playlist2Songs);

    swapPlaylistCounts(&playlist1Songs, &playlist2Songs);

    printf("\nAfter swap:\n");
    printf("Playlist 1 songs: %d\n", playlist1Songs);
    printf("Playlist 2 songs: %d\n", playlist2Songs);

    return 0;
}
```

Output:



The screenshot shows a terminal window with a dark background. At the top, there are three tabs: 'PROBLEMS', 'OUTPUT', and 'PORTS'. Below these is a section labeled '✓ TERMINAL'. The terminal content shows the following commands and output:

```
PS C:\Users\Admin\OneDrive\Desktop\tops\practical> gcc demo5.c -o demo5.exe
PS C:\Users\Admin\OneDrive\Desktop\tops\practical> .\demo5.exe
Before swap:
Playlist 1 songs: 22
Playlist 2 songs: 40

After swap:
Playlist 1 songs: 40
Playlist 2 songs: 22
PS C:\Users\Admin\OneDrive\Desktop\tops\practical> 
```

3) Given an array of 5 order amounts (e.g., Zomato orders), use a pointer to iterate through the array and print each amount along with its memory address.

Hint: Use pointer arithmetic to move to the next element.

Code-

```
#include <stdio.h>

int main() {

    float orderAmounts[5] = {250.50, 480.00, 150.75, 620.25, 99.99};

    float *ptr = orderAmounts; // pointer initialized to the first element

    printf("Zomato Order Amounts:\n\n");

    for (int i = 0; i < 5; i++) {

        printf("Order %d: Amount = %.2f | Address = %p\n", i + 1, *(ptr + i), (void *) (ptr + i));

    }

    return 0;

}
```

Output:

```
PROBLEMS  OUTPUT  PORTS
✓ TERMINAL

PS C:\Users\Admin\OneDrive\Desktop\tops\practical> gcc Demo6.c -o Demo6.exe
PS C:\Users\Admin\OneDrive\Desktop\tops\practical> .\Demo6.exe
Zomato Order Amounts:

Order 1: Amount = 250.50 | Address = 000000223AFFFD40
Order 2: Amount = 480.00 | Address = 000000223AFFFD44
Order 3: Amount = 150.75 | Address = 000000223AFFFD48
Order 4: Amount = 620.25 | Address = 000000223AFFFD4C
Order 5: Amount = 99.99 | Address = 000000223AFFFD50
PS C:\Users\Admin\OneDrive\Desktop\tops\practical> 
```

- 4) Create a function increment Followers(int *followers, int n) that increases each follower count in an array (representing Instagram followers for 5 friends) by 100 using pointer arithmetic, then print the updated counts.

Code-

```
#include <stdio.h>

void incrementFollowers(int *followers, int n) {
    for (int i = 0; i < n; i++) {
        *(followers + i) += 100;
    }
}

int main() {
    int igFollowers[5] = {1200, 850, 3000, 450, 2100};
    int n = 5;

    printf("Before increment:\n");
    for (int i = 0; i < n; i++) {
        printf("Friend %d: %d followers\n", i + 1, igFollowers[i]);
    }

    incrementFollowers(igFollowers, n);

    printf("\nAfter increment (+100 each):\n");
    for (int i = 0; i < n; i++) {
        printf("Friend %d: %d followers\n", i + 1, igFollowers[i]);
    }

    return 0;
}
```

Output:

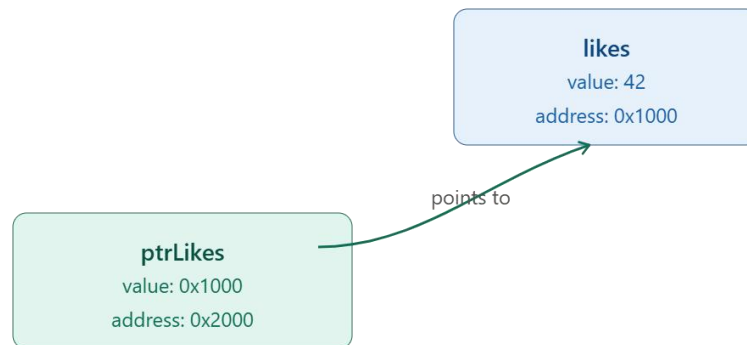
```
PROBLEMS  OUTPUT  PORTS
TERMINAL

PS C:\Users\Admin\OneDrive\Desktop\tops\practical> gcc demo7.c -o demo7.exe
PS C:\Users\Admin\OneDrive\Desktop\tops\practical> .\demo7.exe
Before increment:
Friend 1: 1200 followers
Friend 2: 850 followers
Friend 3: 3000 followers
Friend 4: 450 followers
Friend 5: 2100 followers

After increment (+100 each):
Friend 1: 1300 followers
Friend 2: 950 followers
Friend 3: 3100 followers
Friend 4: 550 followers
Friend 5: 2200 followers
PS C:\Users\Admin\OneDrive\Desktop\tops\practical>
```

5) Draw a simple memory diagram (on paper or using any drawing tool) showing how a pointer references the address of a variable, using the variables from your likes and ptrLikes example; label the variable, pointer, and address.

Diagram:



Read:

- The likes box shows the variable itself — it holds the value 42 and lives at address 0x1000 (a made-up example address, since actual addresses vary each run).
- The ptrLikes box shows the pointer — its own value is 0x1000 (the address of likes), and it lives at its own address, 0x2000.
- The arrow shows the key idea: ptrLikes doesn't store 42 itself — it stores the *address* of likes, which is why we say it "points to" likes.